

Omfattning (scoping)

Med *scoping* avses det område av programkoden där en viss variabel är definierad. En variabel kan inte tilldelas eller refereras på ett ställe i koden där den inte är definierad. Scoping-reglerna för C++ är ganska enkla, men det finns några små fällor.

Lokala variabler

Normalt deklareras variabler i C++ i en viss funktion. Dessa variabler är då *lokala* för denna funktion, och deras scope är således endast denna funktion. Detta gäller även andra block av kod, t.ex. kan man ha lokala variabler i en `if`-sats. De flesta variabler är lokala variabler. Alla variabler som vi använt hittills i våra exempel är lokala variabler. Ett exempel:

```
void exempel (int Parameter) {  
    int Hjalp;  
}  
  
int main () {  
    int Tal;  
    float Area;  
  
    // anropa vår funktion  
    exempel ( Tal );  
}
```

Det enkla "programmet" ovan har tre lokala variabler, nämligen `Tal`, `Area`, `Hjalp` och `Parameter`. De har alla scope endast i sina respektive funktioner, d.v.s. `exempel()` kan inte referera varken `Tal` eller `Area`. Vill vi att en funktion skall kunna referera en viss lokal variabel måste den skickas med som parameter till funktionen i fråga. Även om två funktioner har lokala variabler med samma namn förekommer inga problem, de är helt enkelt skilda variabler med olika scope. Om man i misstag har både en lokal variabel och en parameter med samma namn bör kompilatorn ge ett felmeddelande. Lokala variabler är enkla och naturliga att använda, och vi kommer att använda dem även i alla exempel framöver.

Globala variabler

Motsatsen till lokala variabler är *globala variabler*. En global variabel har scope i hela den fil där den är deklarerad (och även i andra om den definieras så). Detta innebär i praktiken att alla funktioner kan referera den globala variabeln utan att behöva deklarera detta på något sätt. Ett exempel:

```
#include <iostream>  
  
// global variabel  
int Global = 10;  
  
void testaGlobal () {  
    // sätt ett nytt värde på den globala variabeln  
    Global = 20;  
}
```

```
int main () {
    cout << "Global har värdet: " << Global << endl;
    testaGlobal ();
    cout << "Global har värdet: " << Global << endl;
}
```

Om vi kör programmet ser vi att `Global` först har värdet 10 och efter att `testaGlobal()` exekverats har den värdet 20. Globala variabler måste deklarerars före de kan användas. Om vi har ett program som är utspritt över flera filer och vi har en global variabel i en fil A som vi vill referera till i en annan fil B, måste vi deklarerera variabeln som `extern` i B så att kompilatorn vet att det är en global variabel som är definierad i en annan fil. Kör man ovannämnda program får man denna utskrift:

```
% ./GlobalVariable
Global har värdet: 10
Global har värdet: 20
%
```

Statiska variabler

Normala lokala variabler håller sitt värde så länge som de är i scope, d.v.s. normalt så länge den funktion där de är definierade exekveras. Så fort som funktionen eller kodblocket avslutas tappas variabeln sitt värde för att åter bli nollställd nästa gång funktionen anropas. Globala variabler däremot håller sitt värde under hela programmets exekveringstid. Ibland kanske man dock vill ha en lokal variabel som kan hålla sitt värde mellan anrop av funktionen. Man kanske vill hålla ordning på en viss flagga eller något beräknat värde. I dessa fall kan man använda sig av *statiska* variabler. Dessa definieras genom att sätta ordet `static` framför variabelns datatyp då den definieras. Allmänt ser det ut så här:

```
static datatyp variabelnamn;
```

Ett enkelt exempel på en funktion som håller reda på hur många gånger den anropats kan man göra på följande sätt:

```
#include <iostream>

void raknaAnrop () {
    // statisk variabel med värdet 0
    static int Anrop = 0;

    // öka antalet anrop med 1
    Anrop++;
    cout << "raknaAnrop har anropats: " << Anrop << " gånger." << endl;
}

int main () {
    // anropa funktionen 'Anrop' några gånger
    raknaAnrop ();
    for ( int Index = 0; Index < 5; Index++ ) {
        raknaAnrop ();
    }
}
```

Vi initierade `Anrop` till 0 på samma gång som vi deklarerade variabeln. Vill man ha ett initialvärde på en statisk variabel måste det göras på detta sätt, alltså samtidigt som deklarationen.

Ofta används statiska variabler av typen `bool` för att hålla reda på om t.ex. en fil är öppnad, någonting speciellt redan initierats o.s.v. Det kan i så fall se ut som nedan:

```
void lasData () {
    static bool Initierad = false;

    // har vi redan initierat någon resurs, t.ex. en fil?
    if ( ! Initierad ) {
        // initiera resursen
        ...
        // nu har vi initierat resursen
        Initierad = true;
    }

    // gör något med resursen t.ex. läs data
}

int main () {

    // läs data i alla evighet
    while ( true ) {
        lasData ();
    }
}
```

Här har vi nu en boolesk flagga som vi använder för att veta om vi redan initierat någon viss resurs som endast skall initieras en gång. När `lasData()` anropas första gången har `Initierad` värdet `false`, och `if`-satsen utförs således. Efter detta har vi utfört initieringen och flaggan kan sättas till `true`, och `if`-satsen körs aldrig mera.

[Företgående](#)[Parametrar till `main\(\)`](#)[Hem](#)
[Upp](#)[Nästa](#)
[Rekursion](#)